Book Ratings and Popularity Analysis

28753-UWASE Lisa Ornella

Final Project Report — Introduction to Big Data

1. Overview

This project uses book data from the Google Books API to build a complete data pipeline. The main question is whether we can predict a book's average rating or classify it as highly rated using information such as page count, genre, publication year, and number of ratings. The project includes data collection and storage on AWS, data cleaning and analysis, two machine learning models, a Tableau dashboard, and an AI assistant that answers questions about the data.

2. Data Collection and Cleaning

Book data was collected from the Google Books API using 20 genre searches: fiction, history, science, romance, biography, fantasy, mystery, self-help, poetry, philosophy, thriller, young adult, classics, business, cooking, technology, psychology, travel, horror, and sports. This gave us 1,940 unique books after removing duplicates. The raw JSON data was stored in an AWS S3 bucket and then loaded into a PostgreSQL table called books_raw on AWS RDS. The book ID was used to avoid adding the same book more than once, so the pipeline can be run again safely.

The clean table, books_clean, was created by taking the needed fields from the raw JSON and checking the data. Books without a title were removed, although no rows were affected. Page counts of zero or less were treated as missing. Ratings outside the valid 0–5 range were also treated as missing. Publication dates were converted into a simple four-digit year, even when the original date had different formats. Out of the 1,940 books, 1,757 (91%) had no community rating. This is a limitation of the Google Books API because only a small number of books have user ratings. Because of this, the machine learning models were trained using only the 183 rated books, while the full dataset was still used for analysis and the dashboard.

The extraction step also includes retry logic to deal with API rate limits (HTTP 429) and temporary server errors (HTTP 503). This helps the pipeline continue instead of stopping when these errors happen. Database passwords, AWS keys, and API keys were stored in a .env file outside version control. Python logging was used instead of print statements, with INFO, WARNING, and ERROR messages saved with timestamps.

3. Exploratory Data Analysis Findings

The dataset has 1,940 books and no duplicate rows. Fiction is the largest category, with 281 books. Almost all the books are in English: 1,920 out of 1,940, or about 99%. Only about 9% of the books (183) have a community rating. Among the rated books, Science had the highest average rating at 4.70, followed by Juvenile Fiction at 4.41. Biography & Autobiography and Philosophy had the lowest average ratings, at around 3.4. Page count and number of ratings had almost no relationship with the rating. This suggests that genre may be more useful for predicting ratings than these numeric features alone.

4. Machine Learning

4.1 Regression — Predicting Average Rating

Model	MAE	RMSE	R^2
Linear Regression	0.935	1.181	-0.116
Random Forest Regressor (max_depth=6)	0.947	1.240	-0.229

Both models had a negative R^2. This means they did not predict ratings better than simply using the average rating. This result matches the EDA, where page count and popularity showed almost no connection with rating. There were only 146 training rows, and none of the available features had a strong relationship with rating. So this result does not mean the

model was broken. It shows that predicting an exact book rating from this metadata is difficult because important factors such as writing quality, marketing, and review content are not included.

Overfitting Check (Random Forest Regressor)

max_depth	Train R^2	Test R^2
2	0.193	-0.118
6	0.620	-0.229
None	0.837	-0.298

As max_depth increased, the training R^2 went from 0.19 to 0.84, but the test R^2 became worse. This is a clear sign of overfitting. The model was learning the small training dataset too closely instead of finding patterns that work on new data. A max_depth of 2 is therefore the safer choice, even though its test performance is still weak, because it performs the least poorly on unseen data.

4.2 Classification — Predicting 'Highly Rated' (avg_rating ≥ 4.0)

Model	Accuracy	Precision	Recall
Logistic Regression (balanced)	0.568	0.789	0.556
Random Forest Classifier (balanced)	0.622	0.810	0.630

The first classifiers, without class balancing, had an accuracy of about 68–70%. However, this was misleading because about 77% of rated books were highly rated, so the models could get high accuracy by predicting 'highly rated' for most books. The confusion matrix showed that the minority class was not being predicted correctly. Using class_weight='balanced' reduced the overall accuracy but made the predictions more useful for both classes. The Random Forest Classifier correctly identified 6 of 10 lower-rated books and 17 of 27 highly rated books in the test set. Overall, Random Forest performed better than Logistic Regression in accuracy, precision, and recall.

Overfitting Check (Random Forest Classifier)

max_depth	Train Accuracy	Test Accuracy
2	0.630	0.541
6	0.801	0.622
None	1.000	0.649

With max_depth=None, the Random Forest had 100% training accuracy but only 65% test accuracy. The 35-point gap shows that the model memorized the training data instead of learning patterns that work well on new data. A max_depth of 6 gives a better balance, with 80% training accuracy and 62% test accuracy, so it was chosen as the final model.

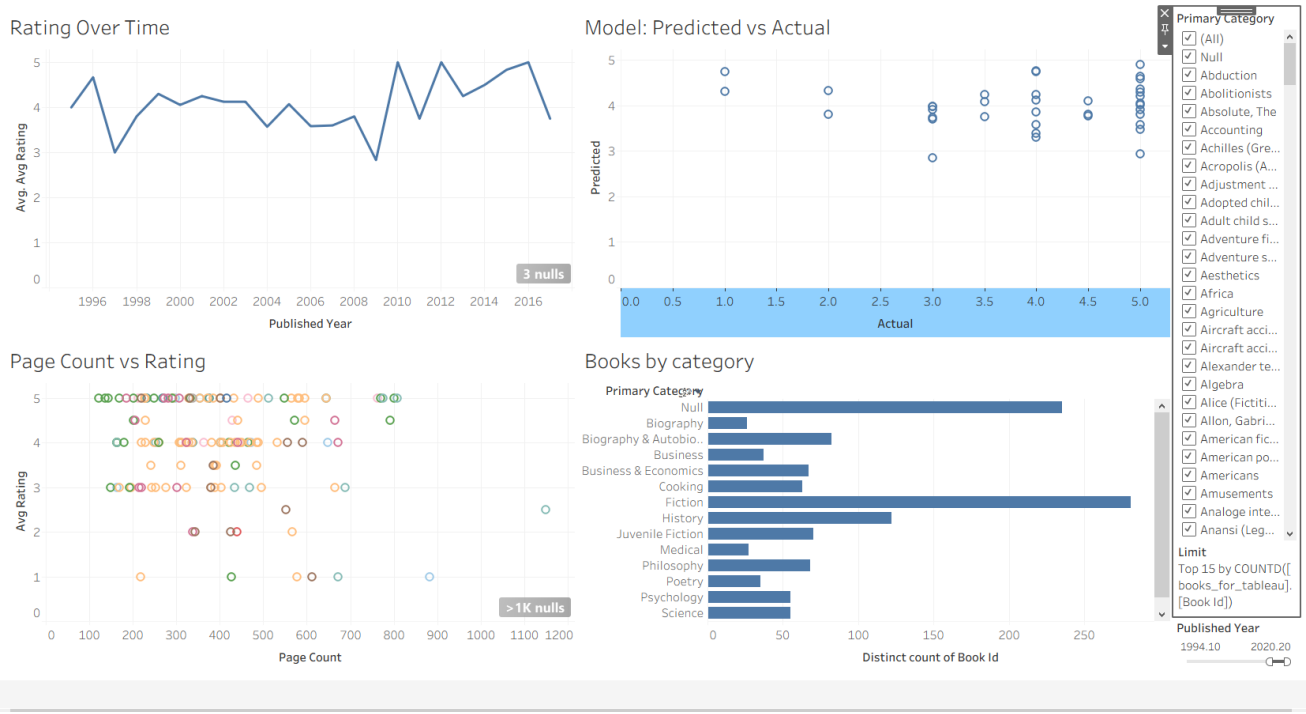
5. Tableau Dashboard

The dashboard has four charts. These are a bar chart showing the number of books in the top 15 categories, a line chart showing average rating by publication year, a scatter plot comparing page count and rating by category, and a scatter plot comparing the Random Forest model's predicted ratings with the actual ratings in the test set. There are also two filters: Primary Category and Published Year range. These filters update the three charts that use the main dataset.

Published dashboard link:

https://public.tableau.com/app/profile/uwase.lisa.ornella/viz/BookRatingsDashboard_17860382044730/BookRatings

Dashboard



6. AI Assistant

The AI assistant is a Python command-line program that connects to the Groq API (Llama 3.3 70B model, free tier). When a user types a plain-English question, the assistant sends the question along with a description of the books_clean table schema to the LLM, which generates a single PostgreSQL SELECT query. A safety check confirms the query is read-only (rejecting any INSERT, UPDATE, DELETE, DROP, ALTER, TRUNCATE, or CREATE statements) before it is executed against the database. The query result is then sent back to the LLM a second time, which converts it into a natural-language sentence for the user.

Example — successful answer:

```
You: Which book has the highest number of ratings?
Assistant: The book "Cat's Cradle" has the highest number of ratings, with 147 ratings.
```

Example — handling a confusing/unanswerable question:

```
You: How many books are set in Paris?
Assistant: Sorry, I couldn't find an answer to that in the data. Could you rephrase your question?
```

In this case, the LLM recognized that 'setting' is not a field in the database. It returned a special NO_QUERY marker instead of making up an answer, and the program gave the user a clear message instead of crashing.

One important bug was also found and fixed. Some queries that grouped books by category or author did not remove NULL values. In some cases, this caused a NULL group to appear as the highest result because of an unusual average. The problem was fixed by telling the model in the schema prompt to remove NULL values from the grouping column before calculating results.

7. Testing

Automated tests were created with pytest for both the data cleaning code and the AI assistant. The data cleaning tests check missing titles, impossible ratings such as 99, negative page counts, valid book records, and extracting a four-digit year from different date formats. The AI assistant tests check that SELECT queries are allowed, DELETE and DROP queries are rejected, unsafe queries are never sent to the database, normal questions return an answer, and unclear questions return a safe message. All 12 tests passed.

8. What I Would Improve With More Time

- Pull more rated books. Only about 9% of Google Books entries have community ratings, which limited the machine learning training set to 183 rows.
- Add more useful features, such as sentiment analysis of book descriptions, to try to improve the model beyond the very low R^2 from metadata alone.